



Intermezzo

**Java Technologies for
Creating a Project**

Project Types

- **Desktop**
- **Web / Enterprise**
- **Mobile**
- **Cloud Computing**
- **Embedded Systems**
- Database
- APIs
- Libraries, ...

Desktop Applications

- **Performance and resource access are critical**
 - High-Performance Computing (HPC)
 - Resource-intensive tasks
- **Offline functionality is essential**
 - Reliability, Data security and privacy
- **Specialized hardware interaction is required**
 - Direct hardware control, OS integration
- **Complex user interfaces are needed**
 - Advanced UI, Specialized user experience
- **Examples:** IDEs (IntelliJ Idea, Netbeans, Eclipse), Creative or industrial design, Security and utilities, etc.

Web Applications

- **Simpler deployment**
 - The application is deployed on a server.
 - Users do not have to download and install software.
- **Cross-platform compatibility**
 - The application can be accessed regardless of the user operating system, using a Web browser.
- **Widespread distribution**
 - Easier to distribute to a large user base, as long as the **scalability** of the application is not an issue.
- **Easier to update and maintain**
 - Updates are deployed on the server-side, ensuring all users have the latest version.

User Interface (UI) / App Logic

- Separating the user interface (UI) from the application logic is a fundamental principle in software engineering. It promotes maintainability, testability, and flexibility
- **Separation of Concerns**
- **MVC / MVVM**
 - Model, View, Control, Data binding, Data flow
- **Frontend / Backend**
- **Monoliths / Microservices**

Core Technologies for Any Project

- **Java SE (Standard Edition)** Core language features, collections, streams, I/O, **JDBC**, **multithreading**, **networking**, **introspection**, **internationalization and localization**, etc.
- **Maven / Gradle** Build automation, dependency management
- **JUnit / TestNG** Unit testing and test automation
- **Lombok** Boilerplate-killer – generates getters/setters, constructors, etc.

The Frontend: Java Desktop GUI Frameworks

- **Swing**: Oldest and most widely used GUI frameworks
 - uses Java2D
- **JavaFX**: Modern and richly featured GUI. It supports 3-D graphics, media streaming, advanced UI types.
 - uses Java 3D
- **SWT** (Standard Widget Toolkit): Uses native OS widgets for a more natural look and feel.
- **JGoodies**: Extends standard Swing components to build professional and polished user interfaces.
- **JIDE**: Enterprise-grade GUI framework with a rich collection of UI components
- ...

The Frontend: Web GUI Frameworks

- **Server-Centric**

- **Java (Jakarta) Server Faces (JSF)**: the standard solution offered by Java Enterprise Edition.
 - Multiple implementations: **PrimeFaces**, Rich Faces, etc.
- **Spring MVC**: uses template-engines, such as Thymeleaf
- **Vaadin**: no need to write JavaScript or HTML
- **Play**: custom solution, very good performance

- **Client-Centric**

- JavaScript frameworks: Angular, React, Vue, etc.
- The **client application (frontend)** must access a **server application (backend)**, providing services for accessing data and other server-side functionalities.

The Backend

- **Data Model**

- Classes that represent application entities.
- Mapping between the classes and DB tables

- **Data Access Layer (Repositories)**

- Repositories provide methods for CRUD (Create, Read, Update, Delete) operations

- **Service Layer**

- Implement business logic in service classes.
- Services interact with repositories and perform data processing.

- **REST APIs (Controllers):**

- Classes that **handle HTTP requests** coming from the Frontend application.

How The Backend Works

In order to really understand what happens on the server side, you need to know the following:

- What is an **application framework**?
- What is an **application server**?
- What is **HTTP** and the **request-response** paradigm?
- How to handle **concurrent** user request?
- How to ensure the **scalability** of the application?
- What is and how to create a DB **connection pool**?
- How to **map classes and tables** in order to access the data?
- What is **Reflection API** and bytecode instrumentation?
- How to **test and deploy** the server-side application.

Spring Framework

The Spring Framework is a powerful, open-source Java framework used to build modern, scalable, and secure applications—from simple web apps to complex enterprise systems and microservices.

- **Spring Boot:** Simplifies the setup and configuration of Spring applications, enabling rapid development and deployment.
 - Spring uses an **application server**: Tomcat, Jetty, ...
- **Spring Core:** Provides fundamental features like dependency injection (DI) and inversion of control (IoC).
- **Spring Data JPA (Java Persistence API):** Abstracts database access and simplifies data persistence
- **Spring REST:** For the creation of RESTful web services.
- **Spring Security:** For securing the services using JWTs.
- **Spring MVC:** For building the Web interface (server-centric).
- **Spring AOP** (Aspect-Oriented Programming), **Spring Test**, ...

Spring is #1 for Distributed Systems

Performance, Maturity, Rich ecosystem, Huge developer community

- Spring is the most used and widely adopted application framework for creating **complex, distributed applications, like large-scale enterprise systems.**
 - Designed to handle big projects with many components.
 - Extensive support for microservices (Spring Cloud)
 - Simple setup with embedded servers
 - Easy to start with: **Spring Initializr** <https://start.spring.io/>
- **Enterprise Adoption:** Used by major companies like Netflix, Amazon, Alibaba, Spotify, large banks, governments, etc.
- **Open Source:** Developed by **Pivotal**, maintained by **VMware** under the **Spring team**: <https://github.com/spring-projects>

What's next for us...

- Graphical User Interfaces (Swing, Java2D, JavaFX)
- Concurrency: Threads and Locks
- Creating Database Applications with JDBC
- Introduction to Java Persistence API (JPA)
- Network Programming (TCP, UDP, HTTP)
- Spring Framework / REST Services
- Class Loading. Reflection API. Annotations
- Internationalization and Localization

Introduction to Java Platform, Enterprise Edition

Concurrency: Threads and Locks

- **Multithreading** is a programming technique that allows multiple threads to run concurrently within a single Java program, sharing the same memory space.
- Perform multiple tasks **concurrently**
- **Improve performance** and responsiveness
 - **Background** processing
 - **Efficient use of CPU** resources, exploiting multiple CPU cores, no idle time during I/O waits
- **Used in** I/O operations, GUIs, servers, and real-time apps
- **Challenges:** Race conditions, Deadlocks, Synchronization, Thread lifecycle management

Working with relational DBs

- **JDBC** (Java Database Connectivity)
 - Raw interaction with databases using SQL
 - `insert` into persons (id, name) values (1, 'John')
 - `select * from persons` //returns **tuples**
- **JPA** (Java Persistence API)
 - Object-relational mapping (ORM) framework.
 - Class *Person* is mapped to table *persons*

```
var person = new Person(1, "John");
entityManager.persist(person);
```
 - `select e from Person e` //returns **objects**

Networking

- **Communication** between **distributed agents**: sending/receiving data, accessing services
- **Protocols**:
 - Internet: **IP**
 - Transport: **TCP, UDP**, etc.
 - Application: **HTTP, JRMP**, FTP, SOAP, etc.
- **Client / Server Model**
- **Web (Application) Server Model**
 - HTTP request / response

RESTful Web Services

- Service implementation over HTTP that conforms to the **REST *architectural style***.
- Data and functionality are considered **resources** and are accessed using an **uniform interface**:
 - **Identification** → **URI**, for example:
`http://example.com/resources`
 - **Representations** → **Standard MIME types**:
`Text, JSON, XML, YAML, etc.`
 - **Operations** → **Standard HTTP methods**:
`HTTP POST, GET, PUT, DELETE`

Reflection API

- Most of the types used by an application are statically specified during design time and their usage is verified by the compiler:

```
java.awt.Button button = new java.awt.Button("Hello");  
button.setBackground(Color.YELLOW);
```

- There are situations when the name of a class, method, field etc. is not known at compile time.

```
String componentName = <read the name of an AWT component>;  
Class clazz = Class.forName(componentName);  
// How do we create the component and set its properties?
```

- *Reflection* - the ability to **inspect** or **use** types at runtime without knowing their names at design-time, instantiate new objects, invoke methods and get/set field values.

i18n & l10n

- **Internationalization** (i18n) is the process of designing an application so that it can be adapted to various languages and regions without engineering changes.
 - Most applications should be designed for international use right from the start of the development.
- **Localization** (l10n) is the process of adapting software for a specific region or language by adding locale-specific components and translating text.
 - Country-specific data should be added without changing the code!